EN4554 – Assignment 2 on CNNs

Name: SULOJAN R.
Index No.: 220626V

GitHub: https://github.com/Sulojan2027/Vision_Coursework/tree/main/Sem07_Deep_Learning_for_Vision/Assignment_2

Contents

- Setup
- Task 1: Backpropagation
- Task 2: Optimizers
- Task 3: Regularization and residual connections
- Task 4: CNN (Optional)
- Conclusion
- References
- Acknowledgement of AI tool use


```
import time
import copy
import random
import math
import os
import json

from google.colab import drive
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torch.utils.data import TensorDataset, DataLoader, random_split
```

```
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
CUDA available: True
GPU: Tesla T4
```

```
SEED = 42

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.manual_seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
set_seed(SEED)
```

```
drive.mount('/content/drive', force_remount=True)
DATA = '/content/drive/MyDrive/datasets/cifar10'
os.makedirs(DATA, exist_ok=True)
torchvision.datasets.CIFAR10(root=DATA, train=True, download=True)
```

```
Mounted at /content/drive
```

```
Dataset CIFAR10
    Number of datapoints: 50000
    Root location: /content/drive/MyDrive/datasets/cifar10
    Split: Train
```

Data loading, split and normalization

```
def get_cifar10_splits(root=DATA, n_train=45_000, n_val=5_000, seed=SEED):
    full_train = torchvision.datasets.CIFAR10(root=root, train=True, download=False)
    test_set = torchvision.datasets.CIFAR10(root=root, train=False, download=False)
    g = torch.Generator().manual_seed(seed)
    train_set, val_set = random_split(full_train, [n_train, n_val], generator=g)
    return train_set, val_set, test_set

def compute_channel_stats(train_subset):
    data = train_subset.dataset.data[train_subset.indices] / 255.0
    return data.mean(axis=(0, 1, 2)), data.std(axis=(0, 1, 2))

train_data, val_data, test_data = get_cifar10_splits()
mean, std = compute_channel_stats(train_data)
print(f"Calculated per channel training statistics: Mean - {mean}, STD - {std}")
```

```
Calculated per channel training statistics: Mean - [0.49165571 0.48233405 0.44667923], STD - [0.24708116
0.24350869 0.26158066]
```

```
MEAN = torch.tensor(mean, dtype=torch.float32).view(1, 3, 1, 1)
STD  = torch.tensor(std,  dtype=torch.float32).view(1, 3, 1, 1)

def to_normalized_tensor(x_uint8):
    x = torch.from_numpy(x_uint8).float().div(255).permute(0, 3, 1, 2)
    return (x - MEAN) / STD

full = train_data.dataset
X_all, y_all = full.data, np.array(full.targets)

X_train = to_normalized_tensor(X_all[train_data.indices])
y_train = torch.from_numpy(y_all[train_data.indices])
X_val   = to_normalized_tensor(X_all[val_data.indices])
y_val   = torch.from_numpy(y_all[val_data.indices])
X_test  = to_normalized_tensor(test_data.data)
y_test  = torch.tensor(test_data.targets)

BATCH_SIZE = 128
train_loader = DataLoader(
    TensorDataset(X_train, y_train), batch_size=BATCH_SIZE, shuffle=True,  pin_memory=True
)
val_loader = DataLoader(
    TensorDataset(X_val, y_val), batch_size=512, shuffle=False, pin_memory=True
)
test_loader = DataLoader(
```

```
    TensorDataset(X_test,  y_test), batch_size=512, shuffle=False, pin_memory=True
)
CLASSES = full.classes
```

Common utilities (training / evaluation loops)

```
def evaluate(model, loader, criterion=None, device=DEVICE):
    """Return (mean_loss, accuracy) over a loader."""
    criterion = criterion or nn.CrossEntropyLoss()
    model.eval()
    total_loss, correct, n = 0.0, 0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
            logits = model(x)
            total_loss += criterion(logits, y).item() * y.size(0)
            correct += (logits.argmax(1) == y).sum().item()
            n += y.size(0)
    return total_loss / n, correct / n

def train_one_epoch(model, loader, optimizer, criterion, augment_fn=None,
                    batch_scheduler=None, device=DEVICE, scaler=None):
    model.train()
    total_loss, correct, n = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        y_true = y if y.ndim == 1 else y.argmax(1)
        if augment_fn is not None:
            x, y = augment_fn(x, y)

        # mixed precision (optional)
        with torch.autocast("cuda", dtype=torch.float16, enabled=scaler is not None):
            logits = model(x)
            loss = criterion(logits, y)
        optimizer.zero_grad(set_to_none=True)
        if scaler is None:
            loss.backward()
            optimizer.step()
        else:
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
        if batch_scheduler is not None:
            batch_scheduler.step()

        total_loss += loss.item() * x.size(0)
        correct += (logits.argmax(1) == y_true).sum().item()
        n += x.size(0)
    return total_loss / n, correct / n

def fit(model, train_loader, val_loader, optimizer, epochs, criterion=None,
        scheduler=None, batch_scheduler=None, augment_fn=None, verbose=True,
        device=DEVICE, amp=False):
    """Train for `epochs`; return history dict of per-epoch lists."""
    criterion = criterion or nn.CrossEntropyLoss()
    model.to(device)
    history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": [],
```

```
                "time": []}
    # amp=True: float16 compute, ~2x faster on T4
    scaler = torch.amp.GradScaler("cuda") if amp else None

    for epoch in range(1, epochs + 1):
        t0 = time.time()
        tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer, criterion,
                                          augment_fn, batch_scheduler, scaler=scaler)
        va_loss, va_acc = evaluate(model, val_loader, criterion)
        if scheduler is not None:
            scheduler.step()
        dt = time.time() - t0

        for k, v in zip(history, (tr_loss, tr_acc, va_loss, va_acc, dt)):
            history[k].append(v)
        if verbose:
            print(f"ep {epoch:3d} | train {tr_loss:.4f} / {tr_acc*100:5.2f}% "
                  f"| val {va_loss:.4f} / {va_acc*100:5.2f}% | {dt:.1f}s")
    return history


def count_parameters(model):
    """Number of trainable parameters."""
    return sum(p.numel() for p in model.parameters() if p.requires_grad)


def plot_history(histories: dict, keys=("train_loss", "val_acc"), title=None):
    """histories: {run_name: history_dict}. One subplot per key, one line per run."""
    fig, axes = plt.subplots(1, len(keys), figsize=(6 * len(keys), 4), squeeze=False)
    for ax, key in zip(axes[0], keys):
        for name, h in histories.items():
            y = np.array(h[key]) * (100 if "acc" in key else 1)
            ax.plot(range(1, len(y) + 1), y, label=name)
        ax.set_xlabel("epoch")
        ax.set_ylabel(key + (" (%)" if "acc" in key else ""))
        ax.set_title(key)
        ax.grid(alpha=0.3)
        ax.legend(fontsize=8)
    if title:
        fig.suptitle(title)
    fig.tight_layout()
    plt.show()
    return fig
```

Task 1: Backpropagation

1(a) Linear, ReLU and softmax cross-entropy with explicit forward / backward (no autograd)

Each layer has a `forward` (caches its input) and a `backward` (returns the gradient w.r.t. its input and stores parameter gradients). $G = \partial L / \partial y$ is the upstream gradient.

- **Linear**, $y = xW + b$: $\dfrac{\partial L}{\partial W} = x^\top G, \quad \dfrac{\partial L}{\partial b} = \sum_i G_i, \quad \dfrac{\partial L}{\partial x} = G W^\top$

- **ReLU**, $y = \max(0, x)$: $\dfrac{\partial L}{\partial x} = G \odot 1[x > 0]$
- **Sigmoid** (used in 1(d)), $s = \sigma(x)$: $\dfrac{\partial L}{\partial x} = G \odot s(1 - s)$
- **Softmax + cross-entropy**, $L = -\frac{1}{N}\sum_i \log p_{i,y_i}$: $\dfrac{\partial L}{\partial z} = \dfrac{1}{N}(p - Y)$, with Y the one-hot labels

Weights use Kaiming initialization, $W \sim \mathcal{N}(0, 2/D_{in})$, $b = 0$.

```
class Linear:
    def __init__(self, in_features, out_features, device=DEVICE, dtype=torch.float32):
        self.W = (torch.randn(in_features, out_features, device=device, dtype=dtype)
                  * np.sqrt(2 / in_features))
        self.b = torch.zeros(out_features, device=device, dtype=dtype)
        self.dW = torch.zeros_like(self.W)
        self.db = torch.zeros_like(self.b)
    def forward(self, x):
        self.x = x
        return x @ self.W + self.b
    def backward(self, grad_out):
        self.dW = self.x.T @ grad_out
        self.db = grad_out.sum(dim=0)
        return grad_out @ self.W.T

    def params(self):
        return [(self.W, self.dW), (self.b, self.db)]

class ReLU:
    def forward(self, x):
        self.mask = x > 0
        return x * self.mask
    def backward(self, grad_out):
        return grad_out * self.mask
    def params(self):
        return []

class Sigmoid:
    def forward(self, x):
        self.s = torch.sigmoid(x)
        return self.s
    def backward(self, grad_out):
        return grad_out * self.s * (1 - self.s)
    def params(self):
        return []

class SoftmaxCrossEntropy:
    def forward(self, logits, targets):
        N = logits.shape[0]
        z = logits - logits.max(dim=1, keepdim=True).values
        log_p = z - torch.logsumexp(z, dim=1, keepdim=True)
        self.p, self.y, self.N = log_p.exp(), targets, N
        return -log_p[torch.arange(N, device=logits.device), targets].mean()
    def backward(self):
        dlogits = self.p.clone()
        dlogits[torch.arange(self.N, device=dlogits.device), self.y] -= 1.0
        return dlogits / self.N
```

```
class MLP:
    def __init__(self, layer_sizes, activation=ReLU, device=DEVICE, dtype=torch.float32):
        self.layers = []
        for in_, out_ in zip(layer_sizes[:-1], layer_sizes[1:]):
            self.layers.append(Linear(in_, out_, device=device, dtype=dtype))
            self.layers.append(activation())
        self.layers.pop()
    def forward(self, x):
        for layer in self.layers:
            x = layer.forward(x)
        return x
    def backward(self, grad):
        for rev_layer in reversed(self.layers):
            grad = rev_layer.backward(grad)
        return grad
    def params(self):
        return [pv for layer in self.layers for pv in layer.params()]
    def linear_layers(self):
        return [l for l in self.layers if isinstance(l, Linear)]
```

1(b) Gradient verification: autograd and centred finite differences

```
def autograd_reference(mlp, x, y):
    """Gradients via autograd"""
    leaves = []
    for k, l in enumerate(mlp.linear_layers()):
        W = l.W.detach().clone().requires_grad_()
        b = l.b.detach().clone().requires_grad_()
        leaves += [W, b]
        x = x @ W + b
        if k < len(mlp.linear_layers()) - 1:
            x = torch.relu(x)
    loss = F.cross_entropy(x, y)
    loss.backward()
    return [t.grad for t in leaves]

def finite_difference_grad(loss_fn, param, eps=1e-5, n_checks=50):
    flat = param.view(-1)
    idx = torch.randperm(flat.numel())[:n_checks]
    grads = torch.zeros(len(idx), dtype=flat.dtype)
    for k, i in enumerate(idx):
        w = flat[i].item()
        flat[i] = w + eps
        L_pos = loss_fn()
        flat[i] = w - eps
        L_neg = loss_fn()
        flat[i] = w
        grads[k] = (L_pos - L_neg) / (2 * eps)
    return grads, idx

def relative_error(a, b):
    return (a - b).norm() / max(a.norm() + b.norm(), 1e-12)
```

```
set_seed()
mlp64 = MLP([3072, 16, 10], dtype=torch.float64)
ce64 = SoftmaxCrossEntropy()
```

```
xb, yb = next(iter(train_loader))
x64 = xb[:8].view(8, -1).to(DEVICE, torch.float64)
y64 = yb[:8].to(DEVICE)

# manual gradients
ce64.forward(mlp64.forward(x64), y64)
mlp64.backward(ce64.backward())
manual = [g.clone() for _, g in mlp64.params()]

# autograd gradients
auto = autograd_reference(mlp64, x64, y64)

# finite differences (subset of entries per parameter)
loss_fn = lambda: ce64.forward(mlp64.forward(x64), y64).item()
names = ["W1", "b1", "W2", "b2"]
rows = []
for name, (p, _), g_man, g_auto in zip(names, mlp64.params(), manual, auto):
    g_fd, idx = finite_difference_grad(loss_fn, p, eps=1e-5, n_checks=50)
    g_man_sub = g_man.view(-1)[idx].cpu()
    rows.append({
        "param": name,
        "shape": tuple(p.shape),
        "rel_err (manual vs autograd)": relative_error(g_man, g_auto).item(),
        "allclose autograd": torch.allclose(g_man, g_auto, rtol=1e-6, atol=1e-8),
        "rel_err (manual vs FD)": relative_error(g_man_sub, g_fd).item(),
        "allclose FD": torch.allclose(g_man_sub, g_fd, rtol=1e-4, atol=1e-6),
    })

with pd.option_context("display.float_format", "{:.2e}".format):   # only for this table
    display(pd.DataFrame(rows))
```

	param	shape	rel_err (manual vs autograd)	allclose autograd	rel_err (manual vs FD)	allclose FD
0	W1	(3072, 16)	0.00e+00	True	8.22e-11	True
1	b1	(16,)	0.00e+00	True	7.48e-11	True
2	W2	(16, 10)	0.00e+00	True	9.84e-11	True
3	b2	(10,)	0.00e+00	True	4.98e-11	True

Interpretation of gradient check

- **Manual vs. autograd:** relative error **0** for all parameters, an exact match.
- **Manual vs. finite differences:** relative error **~5 × 10^{-11} – 10^{-10}**. Finite differences are only an approximation (truncation error ~ε^2 plus rounding), so a tiny error is expected.
- All `torch.allclose` checks are **True**, so the backward pass is correct. float64 is used so that rounding does not hide errors.

1(c) Train the MLP with SGD on flattened CIFAR-10

```
# Hyperparameters
LAYERS = [3072, 512, 256, 10]   # 2 hidden layers
LR, EPOCHS = 0.02, 15           # constant learning rate, batch size = BATCH_SIZE (128)

def evaluate_mlp(mlp, loader):
```

```
    """Forward-only evaluation of the manual MLP -> (mean_loss, accuracy)."""
    loss_ = SoftmaxCrossEntropy()
    total_loss, correct, n = 0.0, 0, 0
    for (x, y) in loader:
        x, y = (x.view(x.size(0), -1)).to(DEVICE), y.to(DEVICE)
        logits = mlp.forward(x)
        total_loss += loss_.forward(logits, y).item() * y.size(0)
        correct += (logits.argmax(1) == y).sum().item()
        n += y.size(0)
    return total_loss / n, correct / n

set_seed()
mlp, ce = MLP(LAYERS), SoftmaxCrossEntropy()
hist = {"iter_loss": [], "train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}

for epoch in range(1, EPOCHS + 1):
    t0 = time.time()
    total_loss, correct, n = 0.0, 0, 0          # reset accumulators every epoch
    for x, y in train_loader:
        x, y = (x.view(x.size(0), -1)).to(DEVICE), y.to(DEVICE)
        logits = mlp.forward(x)
        loss = ce.forward(logits, y)
        mlp.backward(ce.backward())
        for p, g in mlp.params():               # manual SGD step (in place)
            p -= LR * g
        hist["iter_loss"].append(loss.item())
        total_loss += loss.item() * y.size(0)
        correct += (logits.argmax(1) == y).sum().item()
        n += y.size(0)

    val_loss, val_acc = evaluate_mlp(mlp, val_loader)
    hist["train_loss"].append(total_loss / n)
    hist["train_acc"].append(correct / n)
    hist["val_loss"].append(val_loss)
    hist["val_acc"].append(val_acc)
    print(f"ep {epoch:2d} | train {total_loss/n:.4f} / {correct/n*100:5.2f}% "
          f"| val {val_loss:.4f} / {val_acc*100:5.2f}% | {time.time() - t0:.1f}s")
```

```
ep  1 | train 1.7432 / 39.24% | val 1.6020 / 43.82% | 1.3s
ep  2 | train 1.4706 / 48.64% | val 1.5399 / 46.28% | 1.2s
ep  3 | train 1.3523 / 53.03% | val 1.5088 / 46.92% | 1.2s
ep  4 | train 1.2647 / 56.36% | val 1.4861 / 47.92% | 1.2s
ep  5 | train 1.1879 / 59.38% | val 1.4674 / 49.54% | 1.1s
ep  6 | train 1.1223 / 61.64% | val 1.4391 / 50.00% | 1.1s
ep  7 | train 1.0591 / 64.02% | val 1.4341 / 50.56% | 1.4s
ep  8 | train 1.0052 / 66.15% | val 1.4393 / 51.00% | 1.2s
ep  9 | train 0.9486 / 68.03% | val 1.4494 / 51.08% | 1.1s
ep 10 | train 0.8943 / 70.29% | val 1.5137 / 49.66% | 1.2s
ep 11 | train 0.8461 / 71.91% | val 1.4838 / 51.94% | 1.1s
ep 12 | train 0.8004 / 73.50% | val 1.4742 / 51.88% | 1.2s
ep 13 | train 0.7470 / 75.27% | val 1.5415 / 50.34% | 1.2s
ep 14 | train 0.7029 / 77.14% | val 1.5723 / 51.08% | 1.1s
ep 15 | train 0.6607 / 78.72% | val 1.6649 / 49.54% | 1.1s
```

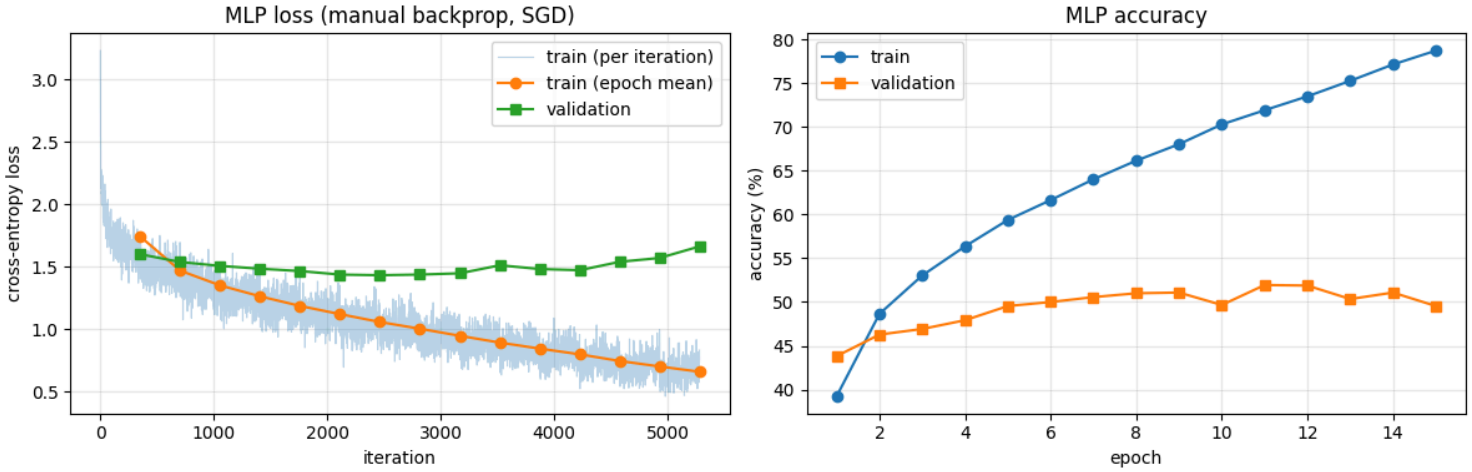
```
iters_per_epoch = len(train_loader)
epochs = np.arange(1, EPOCHS + 1)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(hist["iter_loss"], alpha=0.3, lw=0.8, label="train (per iteration)")
ax1.plot(epochs * iters_per_epoch, hist["train_loss"], "o-", label="train (epoch mean)")
ax1.plot(epochs * iters_per_epoch, hist["val_loss"], "s-", label="validation")
```

```
ax1.set_xlabel("iteration"); ax1.set_ylabel("cross-entropy loss")
ax1.set_title("MLP loss (manual backprop, SGD)"); ax1.legend(); ax1.grid(alpha=0.3)

ax2.plot(epochs, np.array(hist["train_acc"]) * 100, "o-", label="train")
ax2.plot(epochs, np.array(hist["val_acc"]) * 100, "s-", label="validation")
ax2.set_xlabel("epoch"); ax2.set_ylabel("accuracy (%)")
ax2.set_title("MLP accuracy"); ax2.legend(); ax2.grid(alpha=0.3)
plt.tight_layout(); plt.show()

best_ep = int(np.argmax(hist["val_acc"]))
print(f"final val acc: {hist['val_acc'][-1]*100:.2f}%  |  "
      f"best val acc: {hist['val_acc'][best_ep]*100:.2f}% (epoch {best_ep + 1})")
print(f"final train acc: {hist['train_acc'][-1]*100:.2f}%  |  "
      f"train-val gap: {(hist['train_acc'][-1] - hist['val_acc'][-1])*100:.2f} pp")
```




```
final val acc: 49.54%  |  best val acc: 51.94% (epoch 11)
final train acc: 78.72%  |  train-val gap: 29.18 pp
```

Interpretation of MLP training

- **Converges:** train loss **1.74** → **0.66**, train accuracy **39%** → **79%**, so the manual backprop works.
- **Overfits:** validation loss is lowest at **epoch 7 (1.43)** and then rises. Validation accuracy plateaus at **~50%** (best **51.9%**), giving a **~29 pp** train–validation gap.
- **Why overfits?:** flattening discards spatial structure, and with no weight sharing the ~**1.7 M** parameters memorize the training set instead of generalizing.
- **Hyperparameters:** a constant learning rate of 0.02 is stable but noisy. Early stopping (~epoch 7–11) or regularization would help more than extra epochs.

1(d) Per-layer gradient norm at initialization in a 10-layer MLP: sigmoid vs. ReLU

```
sizes = [3072] + [256] * 9 + [10]          # 10 Linear layers, hidden width 256

set_seed()
xb, yb = next(iter(train_loader))          # one fixed batch (128 images), reproducible
x = xb.view(xb.size(0), -1).to(DEVICE)
y = yb.to(DEVICE)

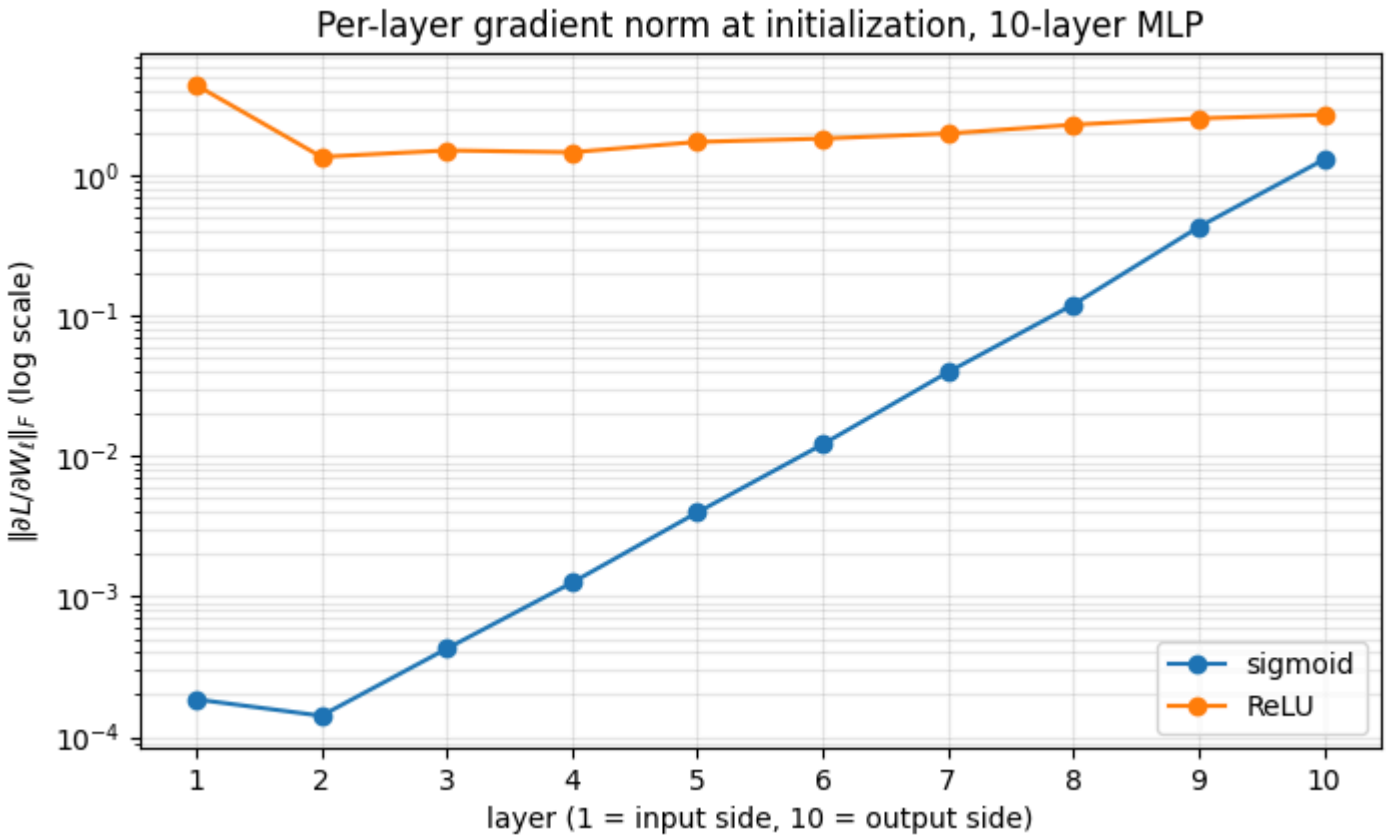
def layer_grad_norms(activation):
    """Per-layer ||dL/dW|| at initialization (one forward/backward, no update)."""
    set_seed()                        # identical initial weights for both activations
    net, ce = MLP(sizes, activation=activation), SoftmaxCrossEntropy()
```

```
    ce.forward(net.forward(x), y)
    net.backward(ce.backward())
    return [l.dW.norm().item() for l in net.linear_layers()]

grad_norms = {
    "sigmoid": layer_grad_norms(Sigmoid),
    "ReLU":    layer_grad_norms(ReLU),
}

layers = np.arange(1, len(sizes))
plt.figure(figsize=(8, 4.5))
for name, norms in grad_norms.items():
    plt.plot(layers, norms, "o-", label=name)
plt.yscale("log")
plt.xticks(layers)
plt.xlabel("layer (1 = input side, 10 = output side)")
plt.ylabel(r"$\|\partial L / \partial W_\ell\|_F$ (log scale)")
plt.title("Per-layer gradient norm at initialization, 10-layer MLP")
plt.legend(); plt.grid(alpha=0.3, which="both")
plt.show()

print(pd.DataFrame(grad_norms, index=[f"L{i}" for i in layers])
      .to_string(float_format="{:.2e}".format))
for name, norms in grad_norms.items():
    print(f"{name:8s} layer1 / layer10 = {norms[0] / norms[-1]:.2e}")
```



```
     sigmoid     ReLU
L1  1.85e-04 4.41e+00
L2  1.41e-04 1.36e+00
L3  4.25e-04 1.51e+00
L4  1.26e-03 1.46e+00
L5  3.98e-03 1.73e+00
L6  1.22e-02 1.83e+00
L7  3.97e-02 1.99e+00
L8  1.21e-01 2.30e+00
L9  4.30e-01 2.55e+00
L10 1.30e+00 2.71e+00
sigmoid  layer1 / layer10 = 1.42e-04
ReLU     layer1 / layer10 = 1.63e+00
```

Explanation: sigmoid vs. ReLU gradients

- **Sigmoid:** the gradient norm shrinks from **1.30** (layer 10) to **1.9 × 10^{-4}** (layer 1), a ratio of **1.4 × 10^{-4}** (**~3×** smaller per layer). Each layer multiplies the backward signal by $\sigma'(z) \leq 0.25$, so it decays exponentially with depth: **vanishing gradients**, and the early layers barely learn.
- **ReLU:** the gradient norm stays between **1.4 and 4.4** in every layer (layer 1 / layer 10 = **1.63**). Its derivative is 1 for active units, and Kaiming initialization keeps the signal scale constant, so all layers get a useful gradient.

Task 2: Optimizers

2(a) SGD with momentum and Adam

Implemented `MySGD` (with optional momentum) and `MyAdam` (with bias correction), then check that they give the same parameters as `torch.optim.SGD` and `torch.optim.Adam` when started from the same weights and trained on the same data.

```python
class MySGD(torch.optim.Optimizer):
    """SGD with optional momentum.
    PyTorch convention: the momentum buffer starts at the first gradient."""
    def __init__(self, params, lr=1e-2, momentum=0.0):
        super().__init__(params, dict(lr=lr, momentum=momentum))

    @torch.no_grad()
    def step(self, closure=None):
        for group in self.param_groups:
            lr, mu = group["lr"], group["momentum"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad
                if mu != 0:
                    state = self.state[p]
                    if "buf" not in state:
                        state["buf"] = g.clone()                # b_1 = g_1
                    else:
                        state["buf"] = mu * state["buf"] + g  # b_t = mu*b_{t-1} + g_t
                    g = state["buf"]
                p -= lr * g
```

```python
class MyAdam(torch.optim.Optimizer):
    """Adam with bias correction."""
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8):
        super().__init__(params, dict(lr=lr, betas=betas, eps=eps))

    @torch.no_grad()
    def step(self, closure=None):
        for group in self.param_groups:
            lr, eps = group["lr"], group["eps"]
            b1, b2 = group["betas"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad
                state = self.state[p]
                if not state:
                    state["t"], state["m"], state["v"] = (0, torch.zeros_like(p),
                                                          torch.zeros_like(p))
                state["t"] += 1
                state["m"] = b1 * state["m"] + (1 - b1) * g          # 1st moment
                state["v"] = b2 * state["v"] + (1 - b2) * g * g      # 2nd moment
                m_hat = state["m"] / (1 - b1 ** state["t"])          # bias correction
                v_hat = state["v"] / (1 - b2 ** state["t"])
                p -= lr * m_hat / (v_hat.sqrt() + eps)
```

```python
xb, yb = next(iter(train_loader))                # one fixed batch, reused for every step
x_eq, y_eq = xb.to(DEVICE), yb.to(DEVICE)

def run(opt_cls, steps=50, **kw):
    set_seed()                       # identical initial weights for both optimizers
    model = nn.Sequential(nn.Flatten(), nn.Linear(3072, 32), nn.ReLU(),
                          nn.Linear(32, 10)).to(DEVICE)
    opt = opt_cls(model.parameters(), **kw)
    for _ in range(steps):
        opt.zero_grad()
        F.cross_entropy(model(x_eq), y_eq).backward()
        opt.step()
    return [p.detach() for p in model.parameters()]

pairs = {
    "SGD":            (MySGD,  torch.optim.SGD,  dict(lr=0.1)),
    "SGD + momentum": (MySGD,  torch.optim.SGD,  dict(lr=0.1, momentum=0.9)),
    "Adam":           (MyAdam, torch.optim.Adam, dict(lr=1e-3)),
}

rows = []
for name, (mine, ref, kw) in pairs.items():
    p_mine, p_ref = run(mine, **kw), run(ref, **kw)
    max_diff = max((a - b).abs().max().item() for a, b in zip(p_mine, p_ref))
    rows.append({
        "optimizer": name,
        "max |Δθ| after 50 steps": f"{max_diff:.2e}",
        "torch.allclose": all(torch.allclose(a, b, rtol=1e-5, atol=1e-6)
                              for a, b in zip(p_mine, p_ref)),
    })
pd.DataFrame(rows)
```

	optimizer	max |Δθ| after 50 steps	torch.allclose
0	SGD	5.96e-08	True
1	SGD + momentum	1.19e-07	True
2	Adam	2.98e-08	True

Interpretation of optimizer equivalence

- After 50 steps from the same weights, all three optimizers match PyTorch: max $|\Delta\theta| \approx$ **10^{-8} – 10^{-7}** and `torch.allclose` is **True**.
- The tiny differences are only **float32 rounding** (machine $\varepsilon \approx 1.2 \times 10^{-7}$), caused by a slightly different order of operations, not by a different algorithm.

2(b) Optimizer trajectories on $f(x, y) = \frac{1}{2}(x^2 + 50y^2)$

```
def f(xy):
    """f(x, y) = 0.5 (x^2 + 50 y^2); works for torch tensors and numpy grids."""
    return 0.5 * (xy[0] ** 2 + 50 * xy[1] ** 2)

START, STEPS = [-10.0, 1.0], 100

def trajectory(opt_cls, **kw):
    xy = torch.tensor(START, requires_grad=True)
    opt = opt_cls([xy], **kw)
    path = [xy.detach().clone()]
    for _ in range(STEPS):
        opt.zero_grad()
        f(xy).backward()
        opt.step()
        path.append(xy.detach().clone())
    return torch.stack(path).numpy()

TRAJ_CFG = {   # SGD is stable only for lr < 2/50 = 0.04; momentum uses the same lr
    "SGD (lr=0.035)":                 (MySGD,  dict(lr=0.035)),
    "SGD + momentum (lr=0.035, 0.9)": (MySGD,  dict(lr=0.035, momentum=0.9)),
    "Adam (lr=0.5)":                  (MyAdam, dict(lr=0.5)),
}
trajectories = {name: trajectory(cls, **kw) for name, (cls, kw) in TRAJ_CFG.items()}

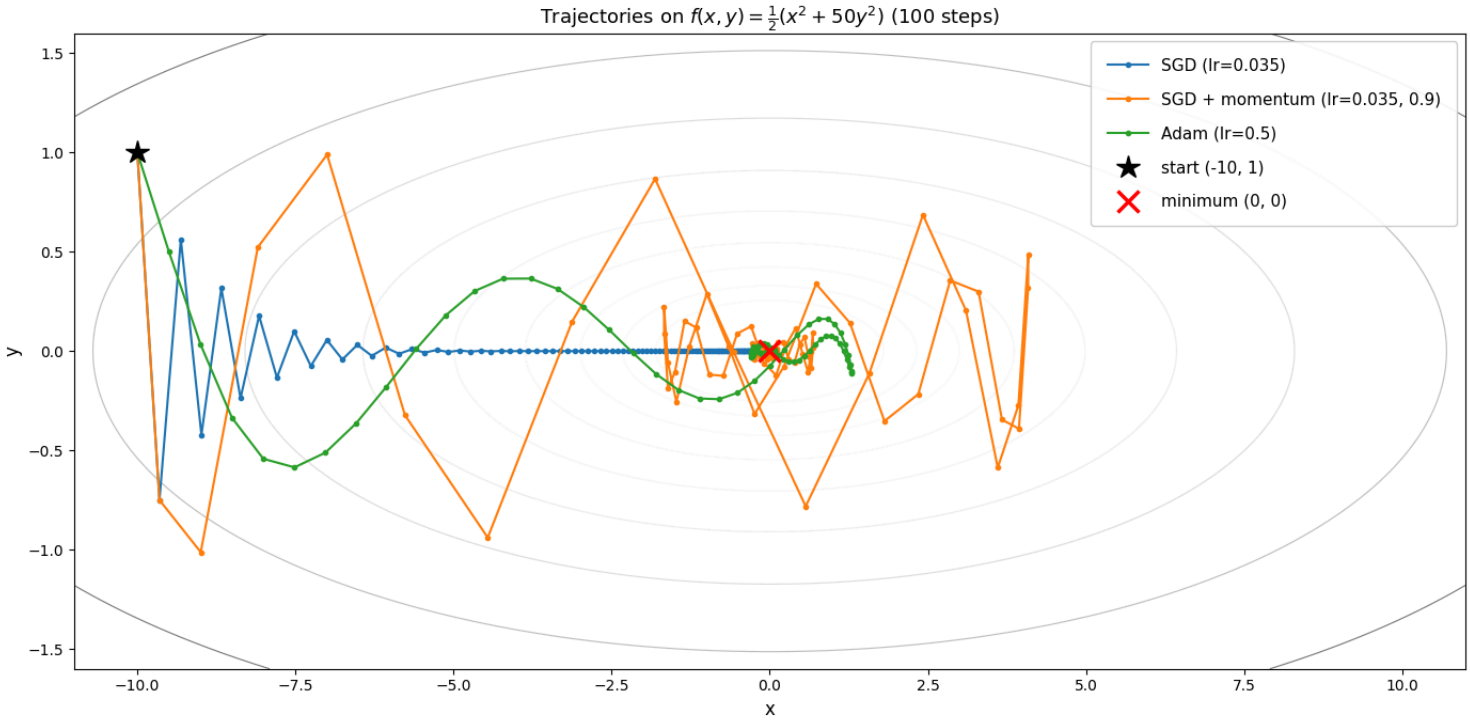
for name, path in trajectories.items():
    fv = np.array([f(p) for p in path])
    hit = np.argmax(fv < 1e-2) if (fv < 1e-2).any() else None
    print(f"{name:32s} final (x, y) = ({path[-1, 0]:+.4f}, {path[-1, 1]:+.4f}) | "
          f"f = {fv[-1]:.2e} | steps to f < 1e-2: {hit}")
```

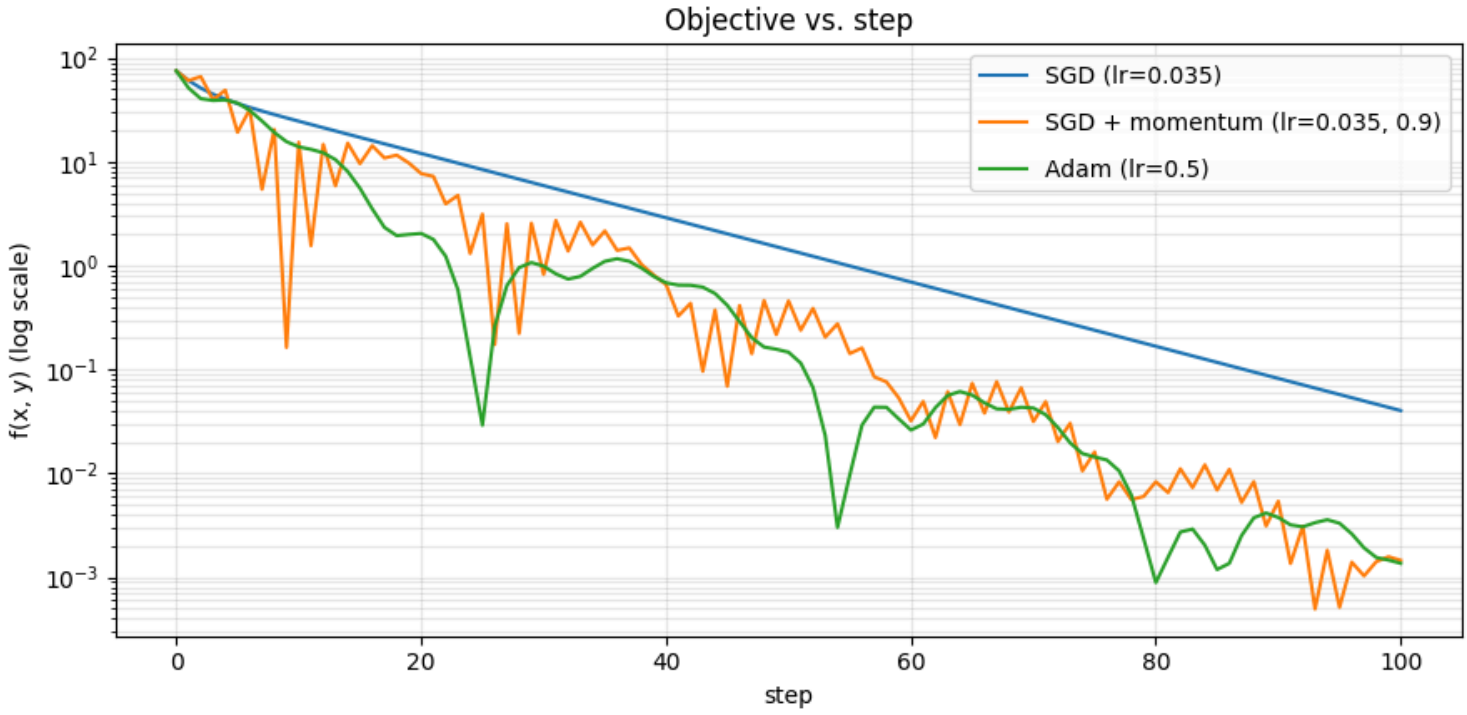
```
SGD (lr=0.035)                   final (x, y) = (-0.2836, +0.0000) | f = 4.02e-02 | steps to f < 1e-2: No
ne
SGD + momentum (lr=0.035, 0.9)   final (x, y) = (-0.0451, +0.0042) | f = 1.45e-03 | steps to f < 1e-2: 76
Adam (lr=0.5)                    final (x, y) = (+0.0421, -0.0043) | f = 1.36e-03 | steps to f < 1e-2: 54
```

```
xs, ys = np.linspace(-11, 11, 400), np.linspace(-1.6, 1.6, 400)
X, Y = np.meshgrid(xs, ys)
Z = f(np.stack([X, Y]))
```

```python
# 1) trajectories on the contour plot
plt.figure(figsize=(14, 7))
plt.contour(X, Y, Z, levels=np.logspace(-2, 2.2, 20), cmap="Greys", linewidths=0.8)
for name, path in trajectories.items():
    plt.plot(path[:, 0], path[:, 1], "o-", ms=3, lw=1.5, label=name)
plt.plot(*START, "k*", ms=16, label="start (-10, 1)")
plt.plot(0, 0, "rx", ms=14, mew=2.5, label="minimum (0, 0)")
plt.xlabel("x", fontsize=12); plt.ylabel("y", fontsize=12)
plt.title(r"Trajectories on $f(x,y)=\frac{1}{2}(x^2+50y^2)$" + f" ({STEPS} steps)",
          fontsize=13)
plt.legend(loc="upper right", fontsize=11, labelspacing=1.1, borderpad=1.0,
           handlelength=2.5, framealpha=0.95)
plt.tight_layout(); plt.show()

# 2) objective value vs. step
plt.figure(figsize=(9, 4.5))
for name, path in trajectories.items():
    plt.plot([f(p) for p in path], lw=1.5, label=name)
plt.yscale("log"); plt.xlabel("step"); plt.ylabel("f(x, y) (log scale)")
plt.title("Objective vs. step"); plt.grid(alpha=0.3, which="both")
plt.legend(labelspacing=1.0)
plt.tight_layout(); plt.show()
```





Explanation of trajectory differences

f is **50× steeper in y than in x**, which makes a long, narrow valley.

- **SGD (lr = 0.035):** the lr must stay below 2/50 = 0.04 for the steep y direction, so the steps along the flat x direction are tiny. It **zig-zags in y** and crawls in x: after 100 steps f = **4 × 10⁻²** (never below 10^{-2}).
- **SGD + momentum (lr = 0.035, μ = 0.9):** velocity builds up along x while the y zig-zags cancel out, so it reaches $f < 10^{-2}$ in **76 steps**. It overshoots a little around the minimum.
- **Adam (lr = 0.5):** it scales each coordinate's step separately, so x and y move at a similar rate. The path is **almost straight** and reaches $f < 10^{-2}$ fastest, in **54 steps**.

2(c) Small CNN with each optimizer at three learning rates

- **Model:** `SmallCNN`, 3 × [Conv3×3 → ReLU → MaxPool] (32/64/128) → FC 256 → FC 10, **620k** parameters, no batch norm.
- **Setup:** 10 epochs, batch 128, same seed for every run.
- **Learning rates:** SGD {0.003, 0.03, 0.3}, SGD + momentum {0.0003, 0.003, 0.03}, Adam {1e-4, 1e-3, 1e-2}. The momentum grid is 10× smaller because momentum 0.9 makes the effective step ~10× larger.

```
class SmallCNN(nn.Module):
    """3 x [Conv3x3 -> ReLU -> MaxPool2] (32/64/128) -> FC 256 -> FC 10, no batch norm."""
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),    # 32 -> 16
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 16 -> 8
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 8 -> 4
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 256), nn.ReLU(),
            nn.Linear(256, num_classes),
        )
```

```
    def forward(self, x):
        return self.classifier(self.features(x))

print(f"SmallCNN parameters: {count_parameters(SmallCNN()):,}")
```

```
SmallCNN parameters: 620,362
```

```
EPOCHS_OPT = 10
GRID = {
    "SGD":            (MySGD,  dict(momentum=0.0), [3e-3, 3e-2, 3e-1]),
    "SGD + momentum": (MySGD,  dict(momentum=0.9), [3e-4, 3e-3, 3e-2]),
    "Adam":           (MyAdam, dict(),             [1e-4, 1e-3, 1e-2]),
}

opt_hist = {}
for name, (cls, kw, lrs) in GRID.items():
    for lr in lrs:
        set_seed()                      # same init + batch order for every run
        model = SmallCNN().to(DEVICE)
        opt = cls(model.parameters(), lr=lr, **kw)
        t0 = time.time()
        opt_hist[(name, lr)] = fit(model, train_loader, val_loader, opt, EPOCHS_OPT,
                                   verbose=False)
        h = opt_hist[(name, lr)]
        print(f"{name:15s} lr={lr:<7g} | final train loss {h['train_loss'][-1]:.4f} "
              f"| best val acc {max(h['val_acc'])*100:5.2f}% | {time.time() - t0:.0f}s")
```

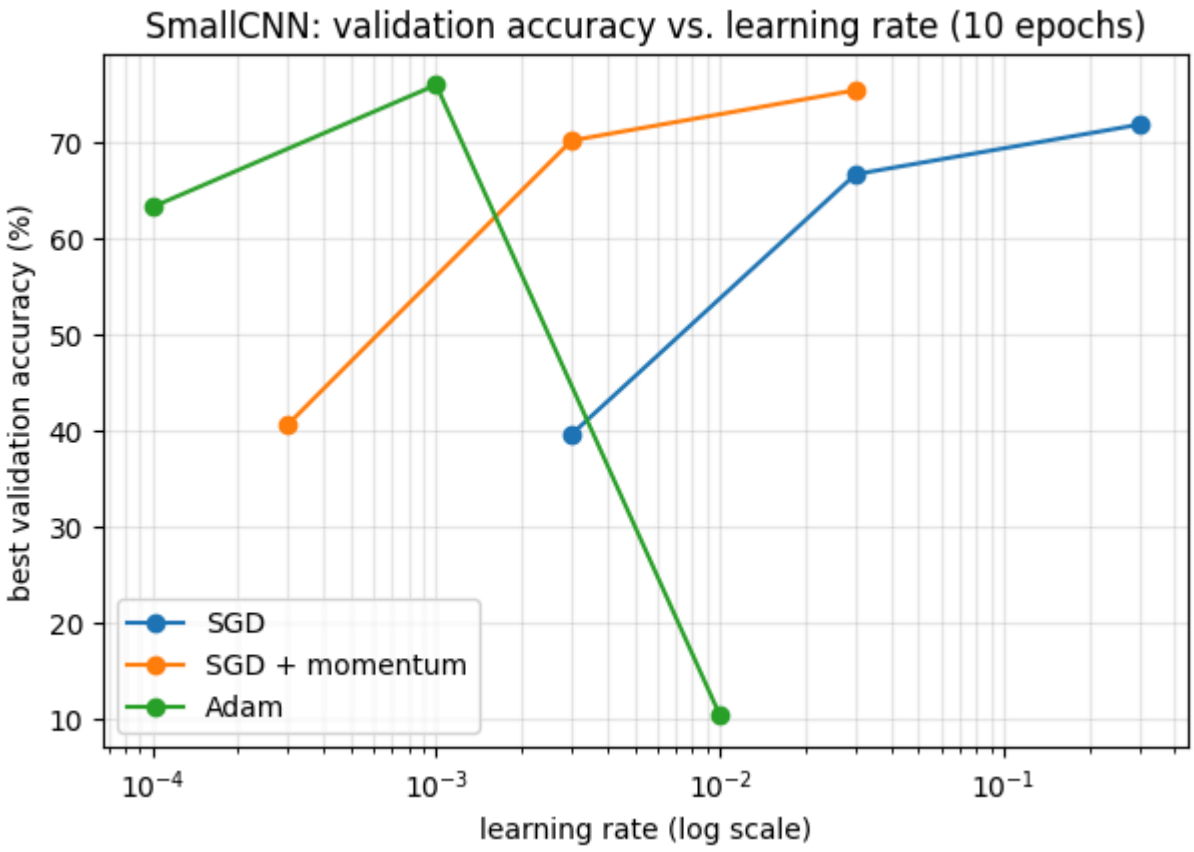
```
SGD             lr=0.003   | final train loss 1.6809 | best val acc 39.56% | 28s
SGD             lr=0.03    | final train loss 0.8132 | best val acc 66.66% | 26s
SGD             lr=0.3     | final train loss 0.2226 | best val acc 71.84% | 26s
SGD + momentum  lr=0.0003  | final train loss 1.6832 | best val acc 40.56% | 26s
SGD + momentum  lr=0.003   | final train loss 0.7655 | best val acc 70.18% | 26s
SGD + momentum  lr=0.03    | final train loss 0.2111 | best val acc 75.42% | 26s
Adam            lr=0.0001  | final train loss 0.9921 | best val acc 63.24% | 28s
Adam            lr=0.001   | final train loss 0.1907 | best val acc 75.98% | 27s
Adam            lr=0.01    | final train loss 2.3034 | best val acc 10.24% | 27s
```

```
best_lr = {}
fig, ax = plt.subplots(figsize=(7, 4.5))
for name, (_, _, lrs) in GRID.items():
    best_val = [max(opt_hist[(name, lr)]["val_acc"]) * 100 for lr in lrs]
    best_lr[name] = lrs[int(np.argmax(best_val))]
    ax.plot(lrs, best_val, "o-", label=name)
ax.set_xscale("log")
ax.set_xlabel("learning rate (log scale)"); ax.set_ylabel("best validation accuracy (%)")
ax.set_title(f"SmallCNN: validation accuracy vs. learning rate ({EPOCHS_OPT} epochs)")
ax.grid(alpha=0.3, which="both"); ax.legend()
plt.show()

plot_history({f"{name} (lr={best_lr[name]:g})": opt_hist[(name, best_lr[name])]
              for name in GRID},
             keys=("train_loss", "val_acc"),
             title="Training at the best learning rate per optimizer")

rows = []
for name, (_, _, lrs) in GRID.items():
    for lr in lrs:
        h = opt_hist[(name, lr)]
        rows.append({
            "optimizer": name,
```

```
            "lr": f"{lr:g}",
            "best val acc (%)": round(max(h["val_acc"]) * 100, 2),
            "final val acc (%)": round(h["val_acc"][-1] * 100, 2),
            "final train loss": round(h["train_loss"][-1], 4),
            "best lr": "✓" if lr == best_lr[name] else "",
        })
pd.DataFrame(rows)
```





	optimizer	lr	best val acc (%)	final val acc (%)	final train loss	best lr
0	SGD	0.003	39.56	39.56	1.6809	
1	SGD	0.03	66.66	66.66	0.8132	
2	SGD	0.3	71.84	68.46	0.2226	✓
3	SGD + momentum	0.0003	40.56	40.56	1.6832	
4	SGD + momentum	0.003	70.18	70.18	0.7655	
5	SGD + momentum	0.03	75.42	74.56	0.2111	✓
6	Adam	0.0001	63.24	63.14	0.9921	
7	Adam	0.001	75.98	75.62	0.1907	✓
8	Adam	0.01	10.24	9.42	2.3034	

Discussion of optimizer sensitivity

optimizer	best val acc at the 3 lrs (low → high)	best lr
SGD	39.6 / 66.7 / **71.8** %	0.3
SGD + momentum	40.6 / 70.2 / **75.4** %	0.03
Adam	63.2 / **76.0** / 10.2 %	0.001

- **Adam is best (76.0%)** and reaches the lowest training loss (**0.19**, vs. 0.21 for momentum and 0.22 for SGD).
- **Adam is the most sensitive to a high lr:** at lr = 0.01 it **diverged** (loss stuck at 2.30 = ln 10, chance-level accuracy). At a low lr it is the most robust (63% vs. ~40%).
- **SGD and momentum fail gracefully:** a low lr just makes them slow. Each momentum run beats the SGD run with a 10× larger lr (40.6 vs. 39.6, 70.2 vs. 66.7, 75.4 vs. 71.8%).
- **Caveat:** only 3 lrs, 10 epochs and 1 seed.

Task 3: Regularization and residual connections

3(a) Regularization study on 5k training images

- **Model:** `SmallBNCNN` (the 2(c) CNN with batch norm), trained on a fixed **5k** subset and validated on the 5k validation split.
- **Setup:** 30 epochs, SGD (lr 0.05, momentum 0.9) with a cosine schedule, same seed for all runs.
- **Configurations:** baseline · weight decay (5e-4) · dropout (0.5) · flip+crop · cutout (8×8) · mixup (α = 0.4) · all combined.

```
def conv_bn(cin, cout):
    return nn.Sequential(nn.Conv2d(cin, cout, 3, padding=1, bias=False),
                         nn.BatchNorm2d(cout), nn.ReLU())
```

```python
class SmallBNCNN(nn.Module):
    """3 x [Conv3x3 -> BN -> ReLU -> MaxPool2] (32/64/128) -> FC 256 -> FC 10.
    Optional dropout before each FC layer."""
    def __init__(self, p_drop=0.0, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            conv_bn(3, 32), nn.MaxPool2d(2),       # 32x32 -> 16x16
            conv_bn(32, 64), nn.MaxPool2d(2),      # 16x16 -> 8x8
            conv_bn(64, 128), nn.MaxPool2d(2),     # 8x8 -> 4x4
        )
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Dropout(p_drop),
            nn.Linear(128 * 4 * 4, 256), nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(256, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))

print(f"SmallBNCNN parameters: {count_parameters(SmallBNCNN()):,}")
```

```
SmallBNCNN parameters: 620,586
```

Augmentations (implemented from scratch, applied batch-wise on the GPU):

```python
# All augmentations: (x, y) -> (x, y) on a normalized GPU batch x of shape (N,3,32,32).

def random_flip_crop(x, y, pad=4):
    """Random horizontal flip + random 32x32 crop from the zero-padded image."""
    N, _, H, W = x.shape
    flip = torch.rand(N, device=x.device) < 0.5
    x = torch.where(flip[:, None, None, None], x.flip(3), x)
    xp = F.pad(x, (pad, pad, pad, pad))
    i = torch.randint(0, 2 * pad + 1, (N,)).tolist()
    j = torch.randint(0, 2 * pad + 1, (N,)).tolist()
    x = torch.stack([xp[k, :, i[k]:i[k] + H, j[k]:j[k] + W] for k in range(N)])
    return x, y

def cutout(x, y, size=8):
    """Zero out one random size x size square per image (may be clipped at the border)."""
    N, _, H, W = x.shape
    cy = torch.randint(0, H, (N, 1, 1), device=x.device)
    cx = torch.randint(0, W, (N, 1, 1), device=x.device)
    ys = torch.arange(H, device=x.device).view(1, H, 1)
    xs = torch.arange(W, device=x.device).view(1, 1, W)
    mask = ((ys - cy).abs() < size // 2) & ((xs - cx).abs() < size // 2)    # (N, H, W)
    return x * (~mask).unsqueeze(1), y

def mixup(x, y, alpha=0.4, num_classes=10):
    """Mix pairs of images and their one-hot labels with lam ~ Beta(alpha, alpha)."""
    lam = float(np.random.beta(alpha, alpha))
    perm = torch.randperm(x.size(0), device=x.device)
    y1h = F.one_hot(y, num_classes).float()
    return lam * x + (1 - lam) * x[perm], lam * y1h + (1 - lam) * y1h[perm]

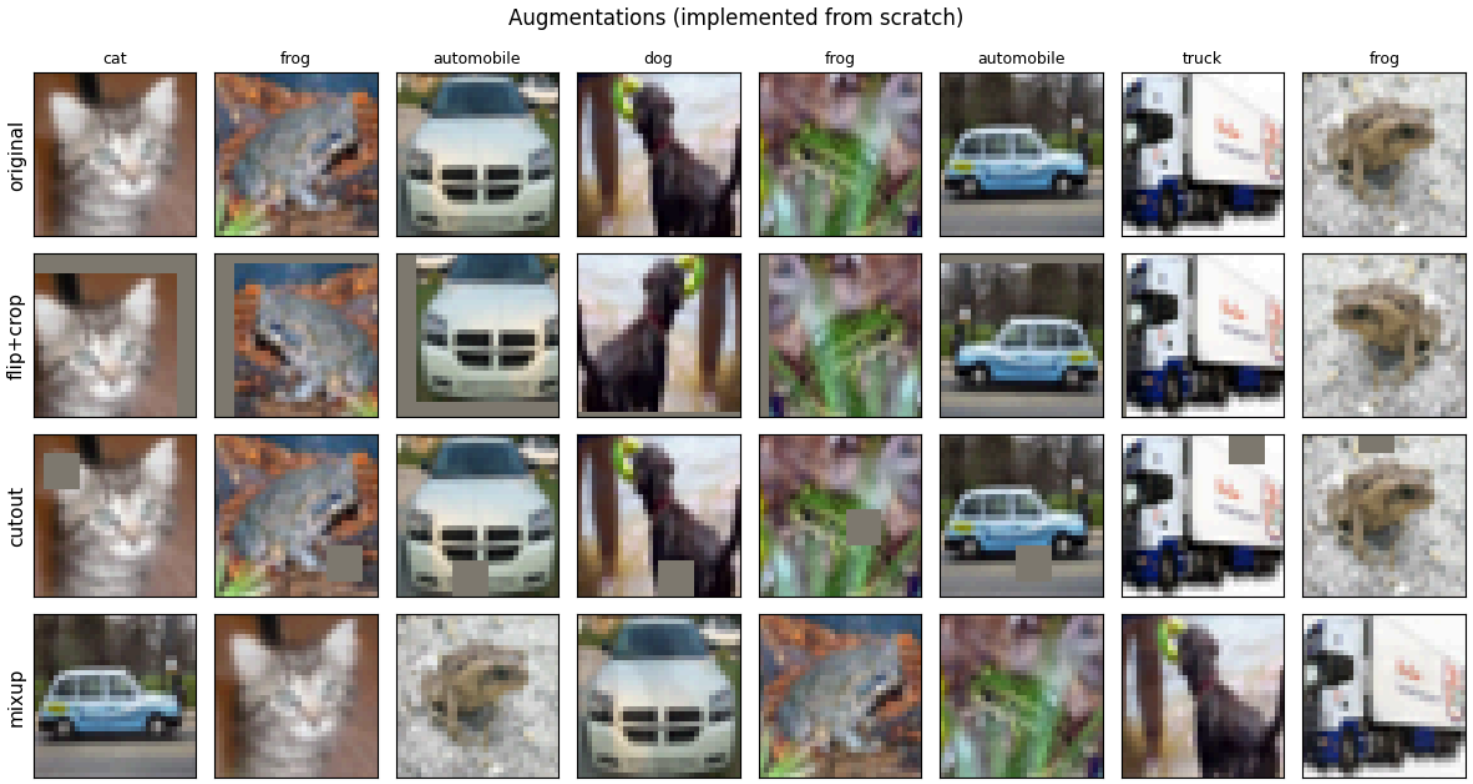
def compose(*fns):
    """Chain augmentations, e.g. compose(random_flip_crop, cutout)."""
    def inner(x, y):
        for fn in fns:
```

```
            x, y = fn(x, y)
        return x, y
    return inner
```

```
set_seed()
xb, yb = next(iter(train_loader))
xb, yb = xb[:8].to(DEVICE), yb[:8].to(DEVICE)

aug_rows = {
    "original":  xb,
    "flip+crop": random_flip_crop(xb, yb)[0],
    "cutout":    cutout(xb, yb)[0],
    "mixup":     mixup(xb, yb)[0],
}
mean_d, std_d = MEAN.to(DEVICE), STD.to(DEVICE)

fig, axes = plt.subplots(len(aug_rows), 8, figsize=(12, 6.5))
for r, (name, imgs) in enumerate(aug_rows.items()):
    imgs = (imgs * std_d + mean_d).clamp(0, 1).cpu()
    for c in range(8):
        axes[r, c].imshow(imgs[c].permute(1, 2, 0))
        axes[r, c].set_xticks([]); axes[r, c].set_yticks([])
    axes[r, 0].set_ylabel(name, fontsize=11)
for c in range(8):
    axes[0, c].set_title(CLASSES[yb[c]], fontsize=9)
plt.suptitle("Augmentations (implemented from scratch)")
plt.tight_layout(); plt.show()
```




```
# Results are saved to Drive after every run, so a Colab disconnect keeps finished runs.
RESULTS_DIR = "/content/drive/MyDrive/a2_results"
os.makedirs(RESULTS_DIR, exist_ok=True)

def run_cached(tag, train_fn):
    """Load the history for `tag` from Drive if it exists,
    otherwise run train_fn() and save it."""
```

```
    path = os.path.join(RESULTS_DIR, f"{tag}.json")
    if os.path.exists(path):
        print(f"[loaded] {tag}")
        with open(path) as fh:
            return json.load(fh)
    h = train_fn()
    with open(path, "w") as fh:
        json.dump(h, fh)
    return h

# fixed 5k training subset (same for every configuration)
g = torch.Generator().manual_seed(SEED)
idx5k = torch.randperm(len(X_train), generator=g)[:5000]
small_loader = DataLoader(TensorDataset(X_train[idx5k], y_train[idx5k]), batch_size=128,
                          shuffle=True, pin_memory=True)
small_eval_loader = DataLoader(TensorDataset(X_train[idx5k], y_train[idx5k]),
                               batch_size=512, shuffle=False)

EPOCHS_REG, LR_REG, WD, P_DROP = 30, 0.05, 5e-4, 0.5
CONFIGS = {   # name: (dropout p, weight decay, augmentation)
    "baseline":     (0.0,    0.0, None),
    "weight decay": (0.0,    WD,  None),
    "dropout":      (P_DROP, 0.0, None),
    "flip+crop":    (0.0,    0.0, random_flip_crop),
    "cutout":       (0.0,    0.0, cutout),
    "mixup":        (0.0,    0.0, mixup),
    "all combined": (P_DROP, WD,  compose(random_flip_crop, cutout, mixup)),
}

def train_reg(p_drop, wd, aug):
    set_seed()
    model = SmallBNCNN(p_drop=p_drop).to(DEVICE)
    opt = torch.optim.SGD(model.parameters(), lr=LR_REG, momentum=0.9, weight_decay=wd)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, EPOCHS_REG)
    h = fit(model, small_loader, val_loader, opt, EPOCHS_REG, scheduler=sched,
            augment_fn=aug, verbose=False)
    # train accuracy on the un-augmented images
    h["clean_train_acc"] = evaluate(model, small_eval_loader)[1]
    return h

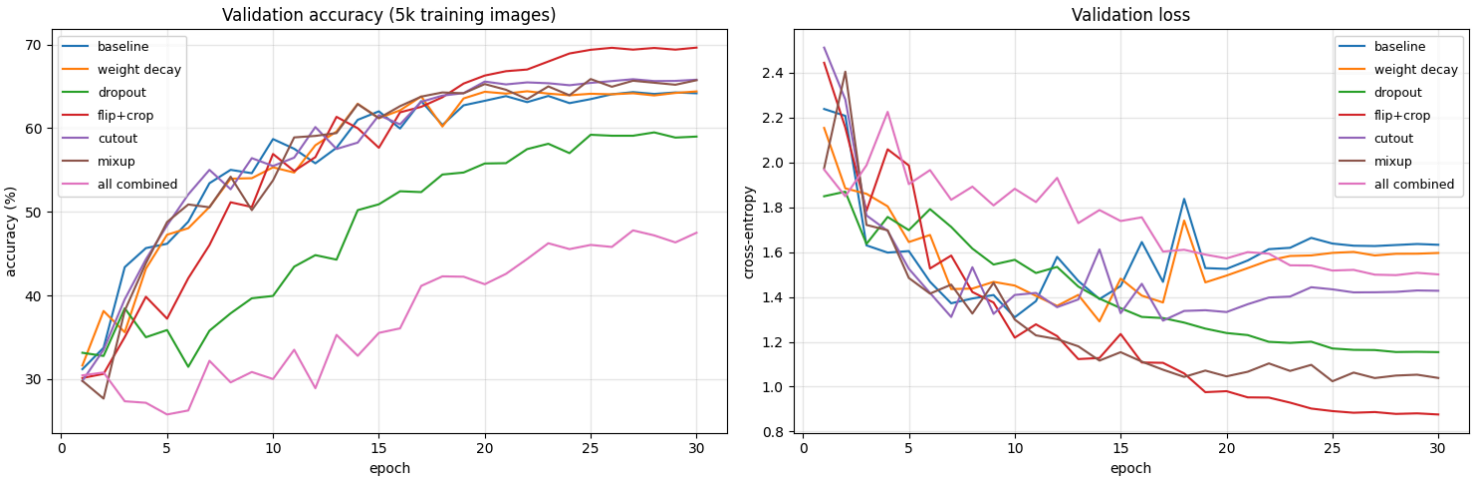
reg_hist = {}
for name, (p, wd, aug) in CONFIGS.items():
    t0 = time.time()
    tag = f"3a_{name.replace(' ', '_').replace('+', '_')}"
    reg_hist[name] = run_cached(tag, lambda: train_reg(p, wd, aug))
    h = reg_hist[name]
    print(f"{name:13s} | train acc {h['clean_train_acc']*100:5.1f}% "
          f"| val acc {h['val_acc'][-1]*100:5.1f}% | {time.time() - t0:.0f}s")
```

```
[loaded] 3a_baseline
baseline      | train acc  99.9% | val acc  64.2% | 0s
[loaded] 3a_weight_decay
weight decay  | train acc  99.9% | val acc  64.4% | 0s
[loaded] 3a_dropout
dropout       | train acc  68.9% | val acc  59.0% | 0s
[loaded] 3a_flip_crop
flip+crop     | train acc  79.8% | val acc  69.6% | 0s
[loaded] 3a_cutout
cutout        | train acc  99.7% | val acc  65.8% | 0s
[loaded] 3a_mixup
mixup         | train acc  98.2% | val acc  65.7% | 0s
[loaded] 3a_all_combined
all combined  | train acc  48.7% | val acc  47.5% | 0s
```

```python
# Final-epoch numbers; train acc is measured on the clean (un-augmented) 5k images.
rows = []
for name, h in reg_hist.items():
    tr, va = h["clean_train_acc"] * 100, h["val_acc"][-1] * 100
    rows.append({"config": name, "train acc (%)": round(tr, 2),
                 "val acc (%)": round(va, 2), "gap (pp)": round(tr - va, 2)})
display(pd.DataFrame(rows))

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
for name, h in reg_hist.items():
    ep = range(1, len(h["val_acc"]) + 1)
    line, = axes[0].plot(ep, np.array(h["val_acc"]) * 100, label=name)
    axes[1].plot(ep, h["val_loss"], color=line.get_color(), label=name)
axes[0].set_title("Validation accuracy (5k training images)")
axes[0].set_ylabel("accuracy (%)")
axes[1].set_title("Validation loss"); axes[1].set_ylabel("cross-entropy")
for ax in axes:
    ax.set_xlabel("epoch"); ax.grid(alpha=0.3); ax.legend(fontsize=9, labelspacing=0.8)
plt.tight_layout(); plt.show()
```

	config	train acc (%)	val acc (%)	gap (pp)
0	baseline	99.88	64.18	35.70
1	weight decay	99.90	64.40	35.50
2	dropout	68.86	59.00	9.86
3	flip+crop	79.80	69.64	10.16
4	cutout	99.66	65.80	33.86
5	mixup	98.24	65.74	32.50
6	all combined	48.68	47.50	1.18



Interpretation of regularization results

Train accuracy is measured on the clean (un-augmented) 5k images at the final epoch.

- **Baseline overfits: 99.9%** train vs. **64.2%** val, a **36 pp** gap.
- **Flip+crop works best:** val **69.6%** (+5.4 pp), gap down to **10 pp**, and the lowest val loss (0.88). It adds realistic variation, so the model cannot just memorize 5k images.
- **Cutout and mixup** help a little (val **65.8%** / **65.7%**), but the gap stays large (~33 pp).
- **Weight decay** has almost no effect (64.4%). With batch norm, the conv weights' scale does not matter much, and 30 epochs is short.
- **Dropout (0.5)** shrinks the gap (10 pp) but **lowers** val accuracy (59.0%): it is too strong for this small model, which then underfits.
- **All combined** removes the gap (**1 pp**) but **underfits badly** (47.5% val, still rising at epoch 30). The methods **do not add up**: stacking them is too much regularization for 5k images and 30 epochs.
- **Caveat:** one seed and untuned strengths; each method would need its own tuning.

3(b) Plain vs. residual CIFAR-style networks, depth $6n + 2 \in \{8, 20, 56\}$

- Three stages (16/32/64 channels), global average pooling, batch norm, Kaiming initialization.
- **The only difference is the shortcut:** identity, plus a stride-2 subsample with zero-padded channels when the shape changes (no extra parameters, so plain and residual nets have identical parameter counts).
- **Setup:** 15 epochs, SGD (lr 0.1, momentum 0.9, weight decay 1e-4) with a cosine schedule, no augmentation, same seed for all 6 models.

```
class BasicBlock(nn.Module):
    """conv-BN-ReLU-conv-BN (+ shortcut) -> ReLU.
    Shortcut (residual only): identity; when the shape changes, subsample by 2 and
    zero-pad the new channels (option A of He et al.) - no extra parameters, so plain
    and residual nets are identical except for the addition."""
    def __init__(self, in_ch, out_ch, stride, residual: bool):
        super().__init__()
        self.residual = residual
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        self.stride, self.pad_ch = stride, out_ch - in_ch
```

```
    def shortcut(self, x):
        if self.stride != 1:
            x = x[:, :, ::self.stride, ::self.stride]
        if self.pad_ch > 0:
            x = F.pad(x, (0, 0, 0, 0, 0, self.pad_ch))  # zero-pad the channels
        return x

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.residual:
            out = out + self.shortcut(x)
        return F.relu(out)


class CifarNet(nn.Module):
    """Depth 6n + 2: stem conv + 3 stages x n blocks x 2 convs (16/32/64 ch.) + FC."""
    def __init__(self, n, residual: bool, num_classes=10):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv2d(3, 16, 3, padding=1, bias=False),
                                  nn.BatchNorm2d(16), nn.ReLU())
        blocks, in_ch = [], 16
        for out_ch, stride in [(16, 1), (32, 2), (64, 2)]:   # 32x32 -> 16x16 -> 8x8
            for k in range(n):
                s = stride if k == 0 else 1
                blocks.append(BasicBlock(in_ch, out_ch, s, residual))
                in_ch = out_ch
        self.blocks = nn.Sequential(*blocks)
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(64, num_classes))

        for m in self.modules():                          # Kaiming initialization
            if isinstance(m, (nn.Conv2d, nn.Linear)):
                nn.init.kaiming_normal_(m.weight, mode="fan_out", nonlinearity="relu")
                if m.bias is not None:
                    nn.init.zeros_(m.bias)
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.ones_(m.weight); nn.init.zeros_(m.bias)

    def forward(self, x):
        return self.head(self.blocks(self.stem(x)))
```

```
rows = []
for n in (1, 3, 9):
    for residual in (False, True):
        m = CifarNet(n, residual)
        n_layers = sum(isinstance(l, (nn.Conv2d, nn.Linear)) for l in m.modules())
        assert n_layers == 6 * n + 2
        rows.append({"depth": 6 * n + 2, "family": "residual" if residual else "plain",
                     "weight layers": n_layers, "parameters": f"{count_parameters(m):,}"})
pd.DataFrame(rows)
```

	depth	family	weight layers	parameters
0	8	plain	8	75,290
1	8	residual	8	75,290
2	20	plain	20	269,722
3	20	residual	20	269,722
4	56	plain	56	853,018
5	56	residual	56	853,018

```
# same setup for all 6 models; no augmentation (compare optimization, not regularization)
EPOCHS_RES = 15

def train_res(n, residual):
    set_seed()
    model = CifarNet(n, residual).to(DEVICE)
    opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, EPOCHS_RES)
    return fit(model, train_loader, val_loader, opt, EPOCHS_RES, scheduler=sched,
               verbose=False)

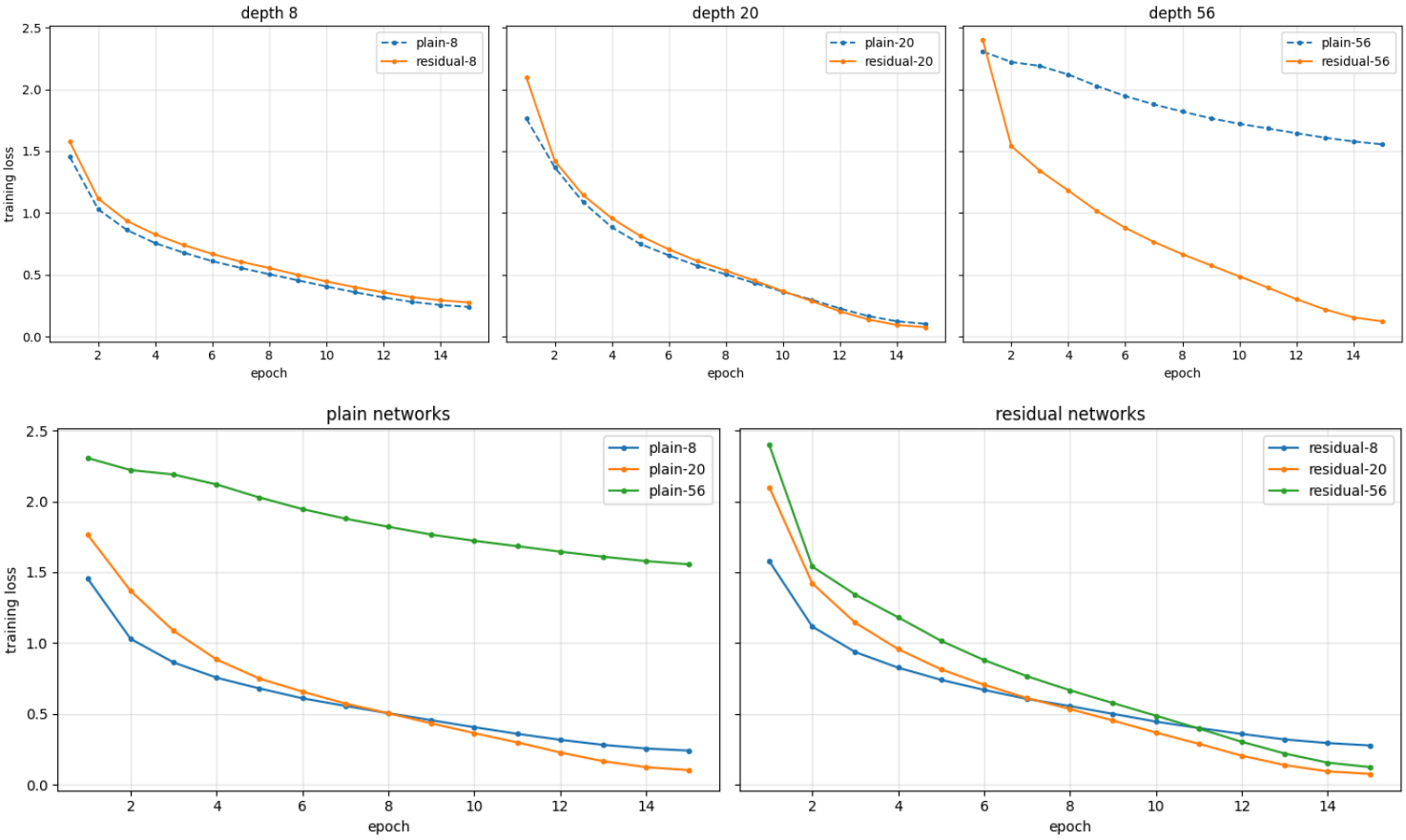
res_hist = {}
for n in (1, 3, 9):
    for residual in (False, True):
        name = f"{'residual' if residual else 'plain'}-{6 * n + 2}"
        t0 = time.time()
        res_hist[name] = run_cached(f"3b_{name}", lambda: train_res(n, residual))
        h = res_hist[name]
        print(f"{name:12s} | final train loss {h['train_loss'][-1]:.4f} "
              f"| final val acc {h['val_acc'][-1]*100:5.2f}% | {time.time() - t0:.0f}s")
```

```
[loaded] 3b_plain-8
plain-8      | final train loss 0.2421 | final val acc 79.90% | 0s
[loaded] 3b_residual-8
residual-8   | final train loss 0.2781 | final val acc 78.86% | 0s
[loaded] 3b_plain-20
plain-20     | final train loss 0.1048 | final val acc 80.80% | 0s
[loaded] 3b_residual-20
residual-20  | final train loss 0.0776 | final val acc 77.34% | 0s
[loaded] 3b_plain-56
plain-56     | final train loss 1.5559 | final val acc 41.60% | 0s
[loaded] 3b_residual-56
residual-56  | final train loss 0.1256 | final val acc 76.80% | 0s
```

```
# training loss: plain vs. residual at each depth
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5), sharey=True)
for ax, depth in zip(axes, (8, 20, 56)):
    ep = range(1, EPOCHS_RES + 1)
    ax.plot(ep, res_hist[f"plain-{depth}"]["train_loss"], "o--", ms=3,
            label=f"plain-{depth}")
    ax.plot(ep, res_hist[f"residual-{depth}"]["train_loss"], "o-", ms=3,
            label=f"residual-{depth}")
    ax.set_title(f"depth {depth}"); ax.set_xlabel("epoch")
    ax.grid(alpha=0.3); ax.legend()
axes[0].set_ylabel("training loss")
plt.tight_layout(); plt.show()
```

```
# same curves grouped by family: does deeper help or hurt?
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5), sharey=True)
for ax, fam in zip(axes, ("plain", "residual")):
    for depth in (8, 20, 56):
        ax.plot(range(1, EPOCHS_RES + 1), res_hist[f"{fam}-{depth}"]["train_loss"], "o-",
                ms=3, label=f"{fam}-{depth}")
    ax.set_title(f"{fam} networks"); ax.set_xlabel("epoch")
    ax.grid(alpha=0.3); ax.legend()
axes[0].set_ylabel("training loss")
plt.tight_layout(); plt.show()

pd.DataFrame([{"model": k, "final train loss": round(h["train_loss"][-1], 4),
               "final train acc (%)": round(h["train_acc"][-1] * 100, 2),
               "final val acc (%)": round(h["val_acc"][-1] * 100, 2)}
              for k, h in res_hist.items()])
```



	model	final train loss	final train acc (%)	final val acc (%)
0	plain-8	0.2421	92.70	79.90
1	residual-8	0.2781	91.43	78.86
2	plain-20	0.1048	97.23	80.80
3	residual-20	0.0776	98.29	77.34
4	plain-56	1.5559	43.59	41.60
5	residual-56	0.1256	96.44	76.80

Explanation: degradation problem and the role of shortcuts

depth	plain: train loss / val acc	residual: train loss / val acc
8	0.24 / 79.9%	0.28 / 78.9%
20	0.10 / 80.8%	0.08 / 77.3%
56	**1.56 / 41.6%**	**0.13 / 76.8%**

- **Shallow nets (8, 20):** plain and residual train almost identically, so shortcuts are not needed yet.
- **Plain-56 degrades:** its **training** loss (1.56) is far worse than plain-8 and plain-20, and it only reaches 43.6% train accuracy. This is **not overfitting** (it cannot even fit the training set); it is an **optimization problem**: the gradient has to pass through 56 layers, so the deep plain net learns very slowly.
- **Residual-56 trains fine:** loss 0.13 and 96.4% train accuracy. The identity shortcut gives the gradient a direct path to the early layers, and each block only has to learn a small change on top of the identity, which is easy.
- **Validation:** the residual nets do not beat plain-20 on validation here (77–79% vs. 80.8%), because with no augmentation and only 15 epochs the deeper models overfit (96–98% train accuracy). The shortcut fixes **training**, not generalization.

Task 4: CNN

Constraints: **≤ 500k parameters**, **≤ 20 training epochs**.

4.1 Design

BestNet, a slim ResNet-9-style network with **~429k parameters** (limit 500k):

stage	layers	output
prep	conv-BN-ReLU (24)	24 × 32 × 32
layer 1	conv-BN-ReLU (48) → max-pool → **residual block** (2 × conv 48)	48 × 16 × 16
layer 2	conv-BN-ReLU (96) → max-pool → **residual block** (2 × conv 96)	96 × 8 × 8
layer 3	conv-BN-ReLU (192) → max-pool	192 × 4 × 4
head	global max-pool → FC 10, logits × 0.125	10

- **Logits × 0.125** keeps the initial outputs small; without it, training at lr 0.1 was unstable (loss spikes to ~45 in a quick synthetic test).
- **Residual blocks** are included because they help deep networks train (Task 3b); the ablation checks whether this shallow 8-conv network needs them. **Batch norm** allows a high learning rate.
- **Width is spent in the later layers** (24 → 192 channels), where the feature maps are small, so the parameter budget goes where it is cheapest to compute.
- **Training recipe:** SGD (Nesterov, momentum 0.9), **one-cycle** lr schedule (max 0.1), weight decay 5e-4, label smoothing 0.1, augmentation **flip+crop + cutout** (the best from Task 3a), batch 128, **20 epochs**.

```
class Residual(nn.Module):
    """x + F(x), where F = two conv-BN-ReLU layers (shortcut=False returns only F(x))."""
    def __init__(self, ch, shortcut=True):
```

```
        super().__init__()
        self.f = nn.Sequential(conv_bn(ch, ch), conv_bn(ch, ch))
        self.shortcut = shortcut

    def forward(self, x):
        return x + self.f(x) if self.shortcut else self.f(x)


class BestNet(nn.Module):
    """Slim ResNet-9-style CNN (~429k parameters)."""
    def __init__(self, c=24, num_classes=10, shortcut=True, logit_scale=0.125):
        super().__init__()
        self.logit_scale = logit_scale   # small initial logits -> stable with lr 0.1
        self.net = nn.Sequential(
            conv_bn(3, c),                                                  # 32x32
            conv_bn(c, 2 * c), nn.MaxPool2d(2), Residual(2 * c, shortcut),     # 16x16
            conv_bn(2 * c, 4 * c), nn.MaxPool2d(2), Residual(4 * c, shortcut), # 8x8
            conv_bn(4 * c, 8 * c), nn.MaxPool2d(2),                         # 4x4
            nn.AdaptiveMaxPool2d(1), nn.Flatten(), nn.Linear(8 * c, num_classes),  # head
        )

    def forward(self, x):
        return self.net(x) * self.logit_scale

n_params = count_parameters(BestNet())
print(f"BestNet parameters: {n_params:,}")
assert n_params <= 500_000
```

```
BestNet parameters: 428,962
```

4.2 Training recipe

```
EPOCHS_BEST = 20                                  # hard limit from the brief
AUG_BEST = compose(random_flip_crop, cutout)

def train_best(tag, shortcut=True, aug=AUG_BEST, wd=5e-4, optimizer="sgd"):
    """Train one BestNet variant; history and weights are cached on Drive."""
    w_path = os.path.join(RESULTS_DIR, f"{tag}.pt")
    model = BestNet(shortcut=shortcut).to(DEVICE)

    def _train():
        set_seed()
        model.load_state_dict(BestNet(shortcut=shortcut).state_dict())     # seeded init
        if optimizer == "sgd":
            opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9,
                                  nesterov=True, weight_decay=wd)
            max_lr = 0.1
        else:
            opt = torch.optim.Adam(model.parameters(), lr=3e-3, weight_decay=wd)
            max_lr = 3e-3
        sched = torch.optim.lr_scheduler.OneCycleLR(
            opt, max_lr=max_lr, epochs=EPOCHS_BEST, steps_per_epoch=len(train_loader))
        h = fit(model, train_loader, val_loader, opt, EPOCHS_BEST,
                criterion=nn.CrossEntropyLoss(label_smoothing=0.1),
                batch_scheduler=sched, augment_fn=aug, amp=True)  # mixed precision
        torch.save(model.state_dict(), w_path)
        return h
```

```
    h = run_cached(tag, _train)
    model.load_state_dict(torch.load(w_path, map_location=DEVICE))
    return model, h

best_models, best_hist = {}, {}
best_models["full"], best_hist["full"] = train_best("4_full")
h = best_hist["full"]
print(f"full model | best val acc {max(h['val_acc'])*100:.2f}% "
      f"| final val acc {h['val_acc'][-1]*100:.2f}% "
      f"| total time {sum(h['time'])/60:.1f} min")
```

```
[loaded] 4_full
full model | best val acc 90.64% | final val acc 90.64% | total time 1.6 min
```

4.3 Ablation (≥ 3 design choices)

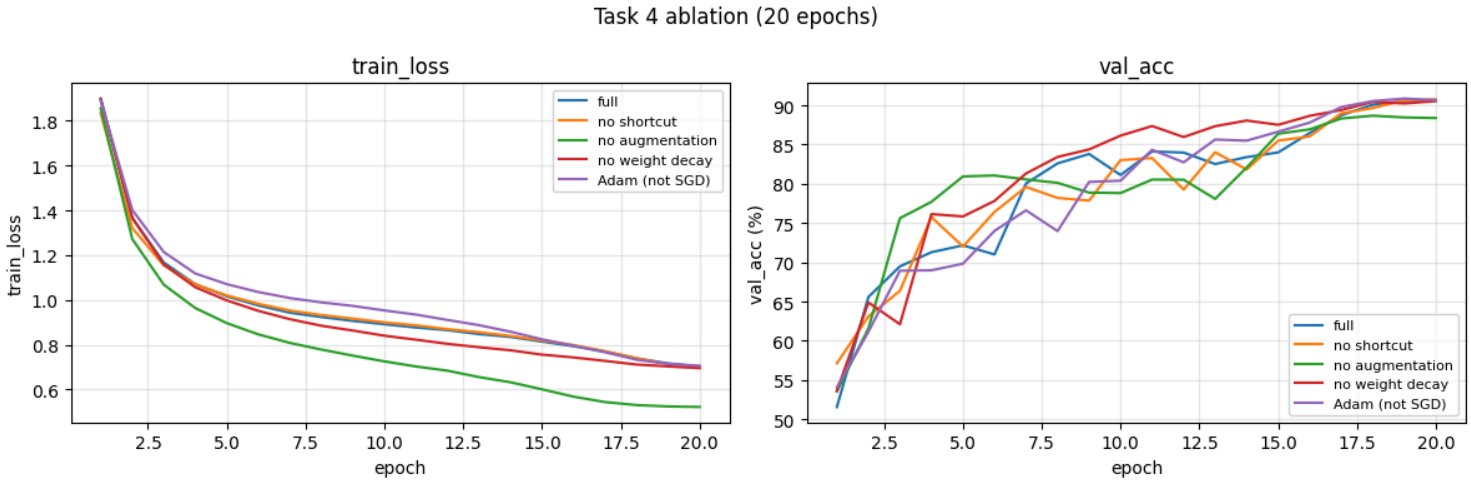
```
# change ONE design choice at a time vs. the full model (same seed, same 20 epochs)
ABLATIONS = {
    "no shortcut":     dict(shortcut=False),
    "no augmentation": dict(aug=None),
    "no weight decay": dict(wd=0.0),
    "Adam (not SGD)":  dict(optimizer="adam"),
}
for name, kw in ABLATIONS.items():
    tag = "4_" + name.replace(" ", "_").replace("(", "").replace(")", "")
    best_models[name], best_hist[name] = train_best(tag, **kw)

full_val = best_hist["full"]["val_acc"][-1] * 100
rows = []
for name, h in best_hist.items():
    rows.append({
        "variant": name,
        "params": f"{count_parameters(best_models[name]):,}",
        "final train acc (%)": round(h["train_acc"][-1] * 100, 2),
        "final val acc (%)": round(h["val_acc"][-1] * 100, 2),
        "Δ val vs. full (pp)": round(h["val_acc"][-1] * 100 - full_val, 2),
    })
display(pd.DataFrame(rows))

_ = plot_history(best_hist, keys=("train_loss", "val_acc"),
                 title="Task 4 ablation (20 epochs)")
```

```
[loaded] 4_no_shortcut
[loaded] 4_no_augmentation
[loaded] 4_no_weight_decay
[loaded] 4_Adam_not_SGD
```

	variant	params	final train acc (%)	final val acc (%)	Δ val vs. full (pp)
0	full	428,962	93.13	90.64	0.00
1	no shortcut	428,962	92.98	90.76	0.12
2	no augmentation	428,962	99.99	88.38	-2.26
3	no weight decay	428,962	93.91	90.54	-0.10
4	Adam (not SGD)	428,962	92.75	90.66	0.02



Discussion of ablation

variant	train acc	val acc	Δ val vs. full
full	93.1%	90.64%	–
no shortcut	93.0%	**90.76%**	+0.12
no augmentation	**100.0%**	**88.38%**	**−2.26**
no weight decay	93.9%	90.54%	−0.10
Adam (not SGD)	92.8%	90.66%	+0.02

- **Augmentation is the only choice that matters:** without it, the model memorizes the training set (**100%** train) and loses **2.3 pp** on validation.
- **Shortcut, weight decay and optimizer make no real difference** (within ±0.12 pp, i.e. single-seed noise). This 8-conv network is shallow enough to train without shortcuts (same as depth 8/20 in Task 3b), and augmentation + label smoothing already regularize it, so weight decay adds little. With a one-cycle schedule, Adam and SGD end up equally good.
- **Model selection used only the validation split:** "no shortcut" has the highest val accuracy, so it was chosen as the final model, although its lead over the full model (0.12 pp) is within noise.

4.4 Final evaluation (test set, once)

```
# Model selection uses ONLY validation accuracy;
# the test set is evaluated once and the result is cached.
final_name = max(best_hist, key=lambda k: best_hist[k]["val_acc"][-1])
final_model = best_models[final_name]
assert count_parameters(final_model) <= 500_000
assert len(best_hist[final_name]["val_acc"]) <= 20

test_path = os.path.join(RESULTS_DIR, "4_test_result.json")
if os.path.exists(test_path):          # never evaluate the test set a second time
    with open(test_path) as fh:
        test_res = json.load(fh)
else:
    test_loss, test_acc = evaluate(final_model, test_loader)
    test_res = {"model": final_name, "test_loss": test_loss, "test_acc": test_acc}
    with open(test_path, "w") as fh:
        json.dump(test_res, fh)
```

```
print(f"final model : {test_res['model']} "
      f"({count_parameters(final_model):,} params, {EPOCHS_BEST} epochs)")
print(f"val acc     : {best_hist[test_res['model']]['val_acc'][-1]*100:.2f}%")
print(f"TEST acc    : {test_res['test_acc']*100:.2f}%   (evaluated once)")
```

```
final model : no shortcut (428,962 params, 20 epochs)
val acc     : 90.76%
TEST acc    : 90.47%   (evaluated once)
```

Final result

- **Test accuracy: 90.47%** (evaluated once) with **428,962 parameters** and **20 epochs**, vs. **90.76%** on validation.
- The small **0.3 pp** val–test gap shows that choosing the model on the validation split did not overfit it; the validation accuracy is a reliable estimate.
- For comparison: MLP ~50% (Task 1), small CNN ~76% (Task 2), plain/residual nets of depth 8–56 ~77–81% without augmentation (Task 3b).

Conclusion

- **Task 1:** manual backprop matches autograd exactly and finite differences to ~10^{-10}. The MLP reaches only **~52%** val and overfits (29 pp gap). Sigmoid gradients **vanish** (~10^{-4} at layer 1) while ReLU + Kaiming init keeps them ~constant.
- **Task 2:** our SGD/momentum/Adam match PyTorch to float32 precision. On the ill-conditioned quadratic, momentum and Adam beat plain SGD's zig-zag. On the CNN, **Adam was best (76.0%)** but most sensitive to a high lr (diverged at 0.01); momentum ≈ SGD with a 10× larger lr.
- **Task 3:** with 5k images, **flip+crop** was the most effective regularizer (+5.4 pp), while stacking all methods underfit. Deep **plain** nets **degrade** (plain-56: train loss 1.56), and **residual** shortcuts fix this optimization problem (residual-56: 0.13).
- **Task 4:** a **429k-parameter** ResNet-style CNN reaches **90.47% test accuracy in 20 epochs**; data augmentation was the most important design choice.

References

1. A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Tech. Rep., Univ. of Toronto, 2009. (CIFAR-10).
2. D. E. Rumelhart, G. E. Hinton, R. J. Williams, "Learning representations by back-propagating errors," *Nature*, 323, 1986.
3. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *JMLR*, 15, 2014.
4. T. DeVries, G. W. Taylor, "Improved Regularization of Convolutional Neural Networks with Cutout," arXiv:1708.04552, 2017.
5. H. Zhang, M. Cisse, Y. N. Dauphin, D. Lopez-Paz, "mixup: Beyond Empirical Risk Minimization," *ICLR*, 2018.
6. K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.
7. D. Page, "How to Train Your ResNet," Myrtle.ai blog, 2018. (ResNet-9 design, logit scaling)
8. PyTorch documentation: `torch.optim.SGD`, `torch.optim.Adam`, `torch.optim.lr_scheduler`, `torchvision.datasets.CIFAR10`, https://pytorch.org/docs

Acknowledgement of AI tool use

This assignment was developed with **Claude (Anthropic)**, used as a coding agent in the VS Code workspace:

- **Notebook skeleton:** Claude generated the section structure and the code skeletons with line-by-line hints.
- **Task:** I implemented these from the hints; Claude reviewed them, fixed bugs.
- **My part throughout:** I ran all experiments on Google Colab (T4 GPU), checked the outputs, and reviewed and edited the code and text.